

StageLink - Plano de Testes Online

Objetivo: validar a aplicacao em condicoes proximas de producao, com utilizadores reais, pagamentos em modo teste, chat WebSocket, uploads, idiomas e administracao.

Ambiente recomendado para teste online: Fly.io, porque o projeto ja tem fly.toml, usa ASGI/Daphne, suporta WebSockets e combina bem com Redis/Upstash. Para media/uploads, usar volume Fly apenas para teste; para producao, preferir S3/Cloudflare R2/Tigris.

1. Preparar ambiente online

Criar app no Fly.io e configurar secrets: SECRET_KEY, DEBUG=False, ALLOWED_HOSTS, CSRF_TRUSTED_ORIGINS, DATABASE_URL, REDIS_URL, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET.

Usar Postgres online para a base de dados. Usar Upstash Redis para Django Channels em producao/teste real.

Criar um volume temporario ou storage externo para uploads: logos, fundo do site, fotos de artistas, galeria, avatar de fas e capas de streams.

Executar migracoes e criar superuser.

2. Contas e permissoes

Criar um admin/superuser e confirmar que entra em qualquer stream sem bilhete nem subscricao.

Criar uma conta Musico e verificar dashboard, edicao de perfil, foto, bio, YouTube, galeria e criacao de streams.

Criar uma conta Fa e verificar avatar circular, edicao de perfil, acesso bloqueado antes de pagar e acesso liberado depois de bilhete/subscricao.

Testar logout/login e login social Google/Facebook se as Social Applications estiverem configuradas.

3. Conteudo e experiencia publica

No admin, configurar logotipo, imagem de fundo e overlay.

No perfil do artista, adicionar foto principal, bio e pelo menos tres fotos de galeria.

Criar streams: um ativo, um futuro e um passado. Cada stream deve ter capa 16:9 e link YouTube valido com embed permitido.

Verificar home, pagina publica do artista, cards de stream e sala de stream em desktop e mobile.

4. Sala de stream e chat

Entrar como admin: deve ver a sala sem pagamento.

Entrar como fa sem acesso: deve ver pagina de acesso/bilhete.

Entrar em duas janelas com utilizadores diferentes e confirmar mensagens em tempo real.

Confirmar que o WebSocket bloqueia utilizadores sem acesso.

Testar video YouTube proprio: publico ou nao listado, sem restricao de idade/copyright, com incorporacao permitida.

5. Pagamentos Stripe em modo teste

Configurar Stripe test keys e webhook publico apontado para /pagamentos/stripe/webhook/.

Testar bilhete individual com cartao 4242 4242 4242 4242.

Testar gorjeta one-time durante stream.

Testar subscricao mensal a artista.

Confirmar no dashboard do musico: subscritores ativos e total de gorjetas.

Confirmar que webhook cria acesso apos pagamento concluido.

6. Idiomas

Testar seletor PT, EN, FR, DE e ES.

Verificar home, login, registo, acesso ao stream, sala de stream e mensagens do player/chat.

Confirmar que nao ha caracteres quebrados como ? em acentos.

7. Checklist final

Home carrega sem erro.

Admin carrega e permite configurar aparencia.

Uploads aparecem depois de refresh.

Chat funciona entre duas sessoes.

Video toca dentro da pagina sem obrigar a abrir YouTube.

Pagamentos teste redirecionam corretamente e webhooks atualizam a base.

Admin entra em todos os streams.

Fa so entra com bilhete ou subscricao.

Logs do servidor nao mostram erros 500.

Servidor recomendado

Sugestao principal: Fly.io para o primeiro teste online, porque esta aplicacao ja esta preparada para ele e precisa de ASGI/WebSockets.

Adicionar Upstash Redis para chat real em producao/teste. O InMemoryChannelLayer serve apenas para desenvolvimento local.

Para uploads, usar Fly Volume so em teste simples. Para algo mais serio, usar storage de objetos como Cloudflare R2, S3 ou Tigris.

Alternativa mais simples para prototipo: Render. Mas para este projeto, Fly.io encaixa melhor no que ja foi construido.